

รายงานความมั่นคงปลอดภัยและความคืบหน้าโครงการ

DevSecOps Security Progress Report & Sprint Review (AI PDF Learning Platform)

กลุ่ม: Group 2 (AI Study Companion / AgentAI)

สถานะ: Fixed / ผ่านการทดสอบ 100% (10/10 Test Suites)

สมาชิก: Papon (Lead Developer / DevSecOps)

GitHub Repo: github.com/HIRONA/ai-pdf-learning-platform

1. ข้อมูลโครงการและวัตถุประสงค์ (Project Overview)

ระบบ AI PDF Learning Platform (AgentAI) เป็นแพลตฟอร์มการเรียนรู้ที่เปิดโอกาสให้นักศึกษาอัปโหลดเอกสารประกอบการเรียน (PDF) ขนาดใหญ่สูงสุด 50MB เพื่อวิเคราะห์ สรุปเนื้อหา ถาม-ตอบอัจฉริยะแบบเรียลไทม์ (Contextual Chat Streaming) และสร้างเครื่องมือช่วยจำ เช่น แบบทดสอบ (Quiz), บัตรคำ (Flashcard) และตารางอ่านหนังสือ (Study Schedule) โดยเชื่อมต่อกับโมเดลปัญญาประดิษฐ์ชั้นนำ (Google Gemini, Groq GPT-OSS 120B และ Qwen) พร้อมการแยกจัดเก็บข้อมูลส่วนบุคคลอย่างปลอดภัย

2. มรรนะ Asset สำคัญและการวิเคราะห์ Attack Surface

2.1 มรรนะ Asset สำคัญของระบบ (Critical Assets)

Asset	รายละเอียดและข้อมูลที่เก็บรักษา	ระดับความสำคัญ	ผลกระทบหากรั่วไหล
User Identity & Credentials	ข้อมูลบัญชีผู้ใช้, รหัสผ่านที่เข้ารหัสด้วย Bcrypt (Cost 12), Session JWT Token	CRITICAL	การสวมรอยบัญชี / Account Takeover
Private Stored PDFs	ไฟล์เอกสาร PDF ต้นฉบับที่ผู้ใช้อัปโหลด จัดเก็บในไดเรกทอรีเฉพาะ uploads/	CRITICAL	ข้อมูลข้อสอบ/เอกสารลับรั่วไหล (Data Breach)
Database (PostgreSQL)	ตารางข้อมูล User, Document, Message, LearningTool, LoginLog	CRITICAL	ข้อมูลทั้งระบบถูกดึงหรือแก้ไขผ่าน SQL Injection
AI API Keys	คีย์ลับ GEMINI_API_KEY, GROQ_API_KEY, BAZAARLINK_API_KEY	HIGH	การถูกขโมยโควตาใช้งานจนเกิดค่าใช้จ่ายสูง (Denial of Wallet)
Web Application Server	Next.js 16 Framework, Turbopack Engine และ Route Handlers	HIGH	บริการล่มจากการโจมตี DoS หรือการรันโค้ดอันตราย (RCE)

2.2 การวิเคราะห์พื้นผิวการโจมตี (Attack Surface Analysis)

- Authentication Endpoints (/login, /register):** สุ่มเสี่ยงต่อ Password Brute-force, Credential Stuffing และ SQL Injection ผ่านฟอร์มกรอกข้อมูล
- Document Upload API (/api/upload):** รับไฟล์ Multipart Form Data ขนาดสูงสุด 50MB เสี่ยงต่อ Arbitrary File Upload, Path Traversal (./../) และ Fake MIME Type
- Files Access API (/api/files/[filename]):** เสี่ยงต่อ Broken Object Level Authorization (BOLA/IDOR) ที่ผู้ใช้หนึ่งอาจพยายามดาวน์โหลดไฟล์ของผู้อื่น
- AI Chat & Tool Endpoints (/api/ai, /api/tools):** รับ Input ข้อความ เสี่ยงต่อ Prompt Injection, Resource Exhaustion (DoS) และ Rate Limiter Bypass
- Admin Console & API (/admin, /api/admin/*):** เสี่ยงต่อ Privilege Escalation และ Missing Function Level Access Control จากผู้ใช้ทั่วไป

3. ทะเบียนความเสี่ยงและการวิเคราะห์ช่องโหว่ (Risk Register - OWASP Top 10)

ตารางประเมินความเสี่ยงตามสูตร: **Risk Score = Likelihood (1–5) × Impact (1–5)**
เกณฑ์ระดับความเสี่ยง: Low (1–6), Medium (8–12), High (14–19), Critical (20–25)

ID	Asset	Threat / Vulnerability	L	I	Risk Level	แนวทางแก้ไข (Mitigation)	Status
R01	Upload API	Arbitrary File Upload & Path Traversal (OWASP A01/A04)	4	5	CRITICAL (20)	ตรวจสอบ Magic Bytes %PDF-, สุ่มชื่อไฟล์ UUID v4, จำกัด Path	FIXED
R02	Files API	Broken Object Level Authorization (IDOR / BOLA) (OWASP A01)	4	5	CRITICAL (20)	ตรวจสอบ Server-side Ownership (userId) ทุก Request ก่อนส่งไฟล์	FIXED
R03	Admin API	Broken Access Control / Privilege Escalation (OWASP A01)	4	5	CRITICAL (20)	เพิ่ม RBAC Guard ที่ Edge Proxy และ Route Handler คืนค่า 403 Forbidden	FIXED
R04	Login Form	Password Brute-Force & Credential Stuffing (OWASP A07)	4	4	HIGH (16)	นับ failedAttempts เมื่อผิดครบ 5 ครั้ง ล็อกบัญชี 30 นาที + บันทึก Log	FIXED
R05	API Endpoints	Rate Limit Bypass ผ่านการปลอมแปลง IP (OWASP A04)	3	4	MEDIUM (12)	ตรวจสอบ Trusted Proxy Policy และใช้ Token-Bucket Rate Limiter	FIXED
R06	Session	Session Hijacking & Insecure Cookie Flags (OWASP A07)	3	4	MEDIUM (12)	บังคับใช้ Cookie Flags httpOnly: true, sameSite: "lax", secure	FIXED
R07	Database	SQL Injection ผ่าน Input Parameters (OWASP A03)	2	5	MEDIUM (10)	ใช้ Prisma ORM ที่สร้าง Parameterized Prepared Statements 100%	FIXED
R08	Packages	Vulnerable Dependency ใน browserslist (OWASP A06)	3	3	MEDIUM (9)	ใช้ npm audit ตรวจสอบ และรัน npm audit fix อัปเดตเวอร์ชัน	FIXED

4. การประยุกต์ใช้เทคโนโลยี/เครื่องมือในวงจร DevSecOps (Tool → Detect → Analyze → Fix)

1. Dependency Scanner: npm audit Detection & Remediation

[Detect]: สแกนพบช่องโหว่ High Severity ใน browserslist ≤4.28.6 (GHSA-c83g-rgw3-j3cx: Unbounded memory growth / Prototype write)
[Analyze]: มีความเสี่ยงต่อการเกิด Denial of Service (Out of Memory) ระหว่างการประมวลผล Build / Compilation
[Fix]: ดำเนินการแก้ไขด้วยคำสั่ง npm audit fix เพื่ออัปเดต dependency graph และ package-lock.json
[Retest]: สแกนซ้ำยืนยันผล found 0 vulnerabilities (ดูหลักฐานใน evidence/sast/npm-audit-report.md)

2. Web Application Security Testing: Burp Suite Dynamic Testing (DAST)

[Detect]: ดักจับ Request ไปยัง GET /api/admin/overview ด้วยสิทธิ์บัญชี STUDENT
[Analyze]: พบช่องโหว่ Broken Access Control โดยระบบเดิมตอบกลับ HTTP 200 OK พร้อมข้อมูลสถิติของแอดมิน เนื่องจากตรวจสอบสิทธิ์เฉพาะที่ UI
[Fix]: ติดตั้ง RBAC Guard ที่ Edge Proxy (src/proxy.ts) และ Route Handler ตรวจสอบบทบาท ADMIN จาก Session Token จริง
[Retest]: ยิง Request เข้าระบบตอบกลับ HTTP 403 Forbidden ทันที (ดูหลักฐานใน evidence/burpsuite/admin-rbac-403.txt)

3. Vulnerability Scanner: OWASP ZAP Automated Baseline Scan

[Detect]: ZAP แจ้งเตือน Missing Security Headers (CSP, nosniff, frame-ancestors) และ Cookie Flags
[Analyze]: ทำให้แอปพลิเคชันมีความเสี่ยงต่อ Clickjacking, MIME Confusion Attacks และ Cookie Sniffing
[Fix]: เพิ่ม Security Headers ครบถ้วนใน next.config.ts และตั้งค่า NextAuth Cookie Flag (httpOnly, sameSite: "lax", secure)
[Retest]: สแกนซ้ำพบว่า Security Headers ถูกต้องครบถ้วนและ Alert ลดลงเหลือศูนย์ (ดูหลักฐานใน evidence/zap/zap-scan-report.md)

4. Automated Security Unit Testing: Node Native Test Runner Regression & SAST Testing

[Detect & Verify]: รันชุดทดสอบความปลอดภัยอัตโนมัติ (tests/security.test.ts และ tests/rbac.test.ts)
[Coverage]: ตรวจสอบ Path Traversal, IP Spoofing, In-memory Rate Limiting, HTTP 413 Oversized Payload และ Regression Auth Bypass
[Result]: ผ่านการทดสอบทั้งหมด 10/10 Test Suites (100% Pass) รับประกันความปลอดภัยในทุก Release

5. หลักฐานการเปรียบเทียบก่อนแก้และหลังแก้ (Before / After Evidence)

จุดที่ 1: การอัปโหลดไฟล์, ตรวจสอบ Magic Bytes และป้องกัน Path Traversal

✗ ก่อนแก้ไข (Vulnerable - เชื้อถือชื่อไฟล์ Client)

```
// เชื้อถือ MIME จาก Browser โดยตรง
if (file.type !== 'application/pdf') return error;

// นำชื่อไฟล์จาก Client ไปต่อ Path ตรงๆ
const filePath = path.join(uploadDir, file.name);
// เสี่ยงต่อ Path Traversal เช่น "../../../package.json"
await fs.writeFile(filePath, buffer);
```

✓ หลังแก้ไข (Secured - ตรวจสอบ Magic Bytes + UUID v4)

```
// ตรวจสอบ Magic Bytes 4 ไบต์แรก (%PDF- / 0x25 0x50 0x44 0x46)
const isRealPdf = buffer[0]===0x25 && buffer[1]===0x50;
if (!isRealPdf) return error('Magic bytes mismatch');

// สุ่มชื่อไฟล์ใหม่เป็น UUID v4 และกั้นบริเวณ Path
const safeFilename = `${crypto.randomUUID()}.pdf`;
const { pdfPath } = resolveStoredDocumentPaths(safeFilename);
await fs.writeFile(pdfPath, buffer);
```

จุดที่ 2: การควบคุมสิทธิ์เข้าถึงหน้าและ API ของผู้ดูแลระบบ (Admin RBAC)

✗ ก่อนแก้ไข (Vulnerable - ขอนปุ่มเฉพาะหน้า UI)

```
// src/components/Navbar.tsx
// ขอนปุ่มเมนูเท่านั้น แต่นักเรียนยิง API ตรงได้
{session?.user.role === 'ADMIN' && (
  Admin Console
)}

// GET /api/admin/overview ไม่มีการตรวจสอบสิทธิ์
export async function GET() { return NextResponse.json(stats); }
```

✓ หลังแก้ไข (Secured - ตรวจสอบสิทธิ์ระดับ Edge Proxy & Route)

```
// src/proxy.ts (Edge Proxy Guard)
if (pathname.startsWith('/api/admin')) {
  if (token?.role !== 'ADMIN') {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }
}

// Server Route Handler ตรวจสอบซ้ำและปฏิเสธด้วย 403
if (!isAdmin(session.user.role)) return forbidden();
```

จุดที่ 3: ระบบป้องกันการเดารหัสผ่าน (Brute-Force Attack & Account Lockout)

✗ ก่อนแก้ไข (Vulnerable - เดารหัสผ่านได้ไม่จำกัด)

```
// ตรวจสอบรหัสผ่านผ่าน Bcrypt ปกติ
const isValid = await bcrypt.compare(pass, user.password);
if (!isValid) {
  // ไม่มีตัวนับความล้มเหลว สามารถยิงสุ่มรหัสผ่านได้เรื่อยๆ
  return null;
}
```

✓ หลังแก้ไข (Secured - ล็อกบัญชี 30s เมื่อผิด 5 ครั้ง)

```
// ตรวจสอบสถานะล็อกบัญชี
if (user.lockedUntil && user.lockedUntil > new Date()) {
  throw new Error(`LOCKED:${remainingSeconds}`);
}

// นับ failedAttempts หากครบ 5 ครั้ง กำหนดเวลาล็อก
const shouldLock = (user.failedAttempts + 1) ≥ 5;
await prisma.user.update({ data: { failedAttempts, lockedUntil } });
await prisma.loginLog.create({ data: { email, ip, success: false } });
```

6. โครงสร้างโฟลเดอร์หลักฐาน (Evidence Folder Structure)

```
ai-pdf-learning-platform/
├── SECURITY_PROGRESS.md           # รายงานสรุปบันทึกทางกรรมน GitHub
├── DevSecOps_Security_Progress_Report.pdf # รายงานฉบับเอกสาร PDF สำหรับส่งงาน
├── evidence/
│   ├── zap/
│   │   └── zap-scan-report.md      # รายงาน Alert และ Headers จาก OWASP ZAP
│   ├── burpsuite/
│   │   ├── admin-rbac-403.txt      # Request/Response บล็อก Admin API ด้วย 403
│   │   ├── idor-document-tampering.txt # Request/Response บล็อกการเข้าถึงเอกสารข้าม User
│   │   └── fake-pdf-magic-bytes.txt # Request/Response บล็อกไฟล์ปลอมด้วย Magic Bytes
│   ├── sast/
│   │   └── npm-audit-report.md      # ผลลัพธ์ npm audit (0 vuln) และ Unit Tests (10/10 Pass)
│   ├── database/
│   │   └── db-security-analysis.md  # วิเคราะห์ Prepared Statements, Bcrypt และ LoginLog
│   └── before-after/
│       ├── point-1-path-traversal-upload.md # เปรียบเทียบโค้ดจุดที่ 1
│       ├── point-2-admin-rbac-authorization.md # เปรียบเทียบโค้ดจุดที่ 2
│       ├── point-3-brute-force-lockout.md   # เปรียบเทียบโค้ดจุดที่ 3
│       └── point-4-idor-and-rate-limiting.md # เปรียบเทียบโค้ดจุดที่ 4
└── src/
```

7. โครงร่างบทพูดสำหรับ Demo Video (5–8 นาที)

ช่วงเวลา	หัวข้อการนำเสนอ	สิ่งที่ต้องแสดงบนหน้าจอและประเด็นพูด
0:00 - 1:00	บทนำและภาพรวมระบบ	เปิดหน้าเว็บจริง (Landing Page & Dashboard) แนะนำชื่อระบบ AI PDF Learning Platform และสมาชิกกลุ่ม 2
1:00 - 2:30	แสดงเครื่องมือและจุดอ่อน	เปิด Burp Suite และจำลองการยิง Request เข้า /api/admin/overview ด้วยสิทธิ์ Student และจำลองการอัปโหลดไฟล์สคริปต์ปลอมนามสกุล .pdf
2:30 - 5:00	แสดงการแก้ไขในโค้ด	เปิด VS Code แสดงฟังก์ชันตรวจ Magic Bytes %PDF-, การสุ่มชื่อ UUID v4, Server Guard ใน src/proxy.ts และผลการรัน npm audit / npm test (10/10 ผ่าน)
5:00 - 7:00	ทดสอบผลลัพธ์บนระบบจริง	ทดสอบเข้าหน้า Admin ด้วยบัญชี Student (ติดหน้า 403 Forbidden), ทดสอบอัปโหลดไฟล์ปลอม (ถูกบล็อก) และทดสอบกรอกรหัสผ่านผิด 5 ครั้ง (หน้าเว็บแสดงเวลานับถอยหลัง 30s)
7:00 - 8:00	สรุปกระบวนการ DevSecOps	สรุปกระบวนการ Tool → Detect → Analyze → Fix และแผนงานนำเข้าสู่ CI/CD Security Pipeline ใน Sprint ถัดไป